K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 4

Unit Title: Linear Data Structures

Assignment Title: Data Management Mini Toolkit (DMMT)

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Shreya Verma

Roll Number: 2501730173

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

INTRODUCTION

This assignment focuses on implementing important non-linear and advanced data structures used in real-world systems. The objective is to understand how data can be stored, searched, and managed efficiently using structures like Binary Search Trees, Graphs, and Hash Tables.

The assignment includes:

- Binary Search Tree (BST) for ordered data storage
- Graph traversal using BFS and DFS
- Hash Table with collision handling using separate chaining

These structures are widely used in databases, networking, caching, and indexing systems.

MODULES IMPLEMENTED

1. BINARY SEARCH TREE (BST)

• Operations Implemented:

- Insert
- Search
- Delete (leaf node, one child, two children)
- Inorder Traversal

• Key Concepts:

- Left subtree contains smaller values
- Right subtree contains larger values

• Time Complexity:

- Average Case: O(log n)
- Worst Case: O(n) (skewed tree)

• Important Observation:

- Inorder traversal prints elements in sorted order because it follows: → Left → Root → Right

2. GRAPH (ADJACENCY LIST + BFS + DFS)

• Representation:

- Graph stored using adjacency list (dictionary-like structure using lists)

• Algorithms Implemented:

BFS (Breadth First Search)

- Uses queue
- Traverses level by level

DFS (Depth First Search)

- Uses recursion
- Traverses depth-wise

• Time Complexity:

- BFS: O(V + E)
- DFS: O(V + E)

• Real-world Applications:

- BFS → shortest path (unweighted graphs)
- DFS → cycle detection, backtracking

3. HASH TABLE (SEPARATE CHAINING)

• Features:

- Key-value storage
- Collision handling using linked lists (chaining)

• Hash Function Used:

index = key % table_size

• Operations:

- Insert
- Get
- Delete

• Time Complexity:

- Average: O(1)
- Worst Case: O(n) (many collisions)

• Collision Explanation:
A collision occurs when multiple keys map to the same index. Separate chaining solves this by storing multiple values in the same bucket.

TESTING & OUTPUT SUMMARY

BST Operations

- Inserted values:
  50, 30, 70, 20, 40, 60, 80
- Search Results:
  20 → Found
  90 → Not Found
- Deletions:
  - Leaf Node → 20
  - One Child → 60
  - Two Children → 30
- Inorder traversal after operations shows correctly sorted data.

GRAPH OUTPUT

• Nodes: A, B, C, D, E, F

• BFS Traversal:
A → B → C → D → E → F

• DFS Traversal:
A → B → D → F → E → C

HASH TABLE OUTPUT

• Table Size: 5

• Inserted Keys:
10, 15, 20, 7, 12

• Collision Observed:

- 10, 15, 20 all mapped to same index

• Retrieval:

- get(10) → success
- get(15) → success
- get(7) → success

• After deleting 15:

- Bucket updated correctly

THEORY VS PRACTICAL OBSERVATION

• BST performs efficiently for searching and sorting when balanced.
• Graph traversal algorithms correctly explore all nodes.
• Hash table provides very fast lookup (almost constant time).

• Observations:

- BST becomes slow if unbalanced
- BFS and DFS both visit all nodes but in different orders
- Hash table performance depends on collision handling

KEY DIFFERENCES

Structure	Purpose	Advantage	Limitation
BST	Ordered data	Fast search	Can become skewed
Graph	Network structure	Flexible relationships	Complex traversal
Hash Table	Fast lookup	O(1) average time	Collisions

CONCLUSION

• Binary Search Tree is useful for maintaining sorted data and fast searching.
• Graphs help in representing real-world networks and solving traversal problems.
• Hash Tables are extremely efficient for quick data retrieval.

Overall, this assignment demonstrates how different data structures are used in real-world applications like databases, networking, and caching systems.

REFERENCES

• Course notes (Data Structures – Unit 4)
• GeeksforGeeks (BST, Graphs, Hashing)
• Python documentation